

Introduction to Data Science and Engineering

- Text mining



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

RB Luo / 羅銳邦

School of Computing and Data Science
University of Hong Kong

Some slides retrofitted by Michael Wu

- 
- Mar 5th
 - Online recording (video available at Moodle), no face-to-face teaching
 - Mar 10th, 12th
 - Reading week `~\_(\`ヾ)_/_`
 - Mar 17th
 - Mid-term in the classroom (MWT2)
 - You need to be physically present to receive a unique code
 - Bring your own laptop
 - **No makeup exam**
 - **0 if without a doctor's note**
 - **With a doctor's note, your mid term grading will be projected as either your project or final exam grading, whichever is worse**



In this lecture, we learn how to generate useful numerical summaries from text data to which we can apply some of the powerful data visualization and analysis techniques we have learned



Todd Vaziri ✓
@tvaziri

Every non-hyperbolic tweet is from iPhone (his staff).

Every hyperbolic tweet is from Android (from him).

Data scientist David Robinson conducted an analysis to determine if data supported this assertion, let's go through his analysis and learn some basic text mining skills

<http://varianceexplained.org/r/trump-tweets/>



Donald J. Trump @realDonaldTrump
Good luck #TeamUSA
#OpeningCeremony #Rio2016
pic.twitter.com/mS8qsQpJPh

27,391 Likes

8,392 Retwee

Aug 5, 2016 at 8:59 PM

via **Twitter for iPhone**



Donald J. Trump @realDonaldTrump
Heading to New Hampshire - will be
talking about Hillary saying her brain
SHORT CIRCUITED, and other thing!

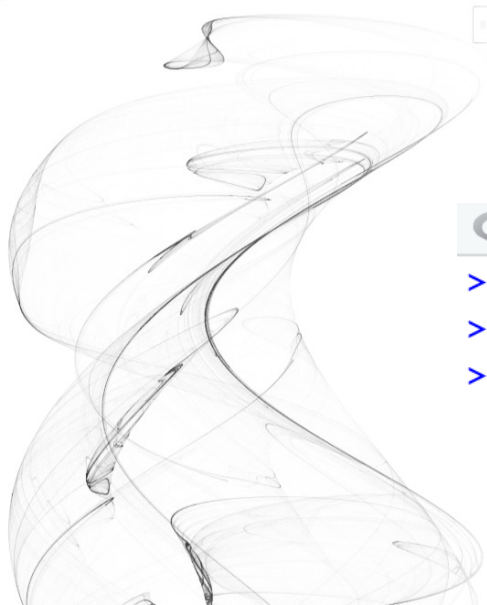
4,451 Likes

1,480 Retwee

Aug 6, 2016 at 11:11 AM

via **Twitter for Android**

<https://twitter.com/tvaziri/status/762005541388378112>



```
R 4.2.2 · ~/ ↵  
> library(tidyverse)  
> library(lubridate)  
> library(scales)
```

```
R 4.2.2 · ~/ ↵  
> url <- 'https://raw.githubusercontent.com/bpb27/trump-tweet-archive/master/data/realdonaldtrump/%s.json'  
> trump_tweets <- map(2009:2017, ~sprintf(url, .x)) |>  
+ map_df(jsonlite::fromJSON, simplifyDataFrame = TRUE) |>  
+ filter(!is_retweet & !str_detect(text, '^\"')) |>  
+ mutate(created_at = parse_date_time(created_at,  
+                                     orders = "a b! d! H!:M!:S! z!* Y!",  
+                                     tz="EST"))  
Date in ISO8601 format; converting timezone from UTC to "EST".
```

```
R 4.2.2 · ~/ ↵  
> str(trump_tweets)  
'data.frame': 20761 obs. of 8 variables:  
 $ source      : chr  "Twitter Web Client" "Twitter Web Client" "Twitter Web Client" "Twitter Web Client" ...  
 $ id_str       : chr  "6971079756" "6312794445" "6090839867" "5775731054" ...  
 $ text         : chr  "From Donald Trump: Wishing everyone a wonderful holiday & a happy, healthy, prosperous  
New Year. Let's think li"| __truncated__ "Trump International Tower in Chicago ranked 6th tallest building in world by Co  
uncil on Tall Buildings & Urban "I __truncated__ "Wishing you and yours a very Happy and Bountiful Thanksgiving!" "Donald  
Trump Partners with TV1 on New Reality Series Entitled, Omarosa's Ultimate Merger: http://tinyurl.com/yk5m3lc" ...  
 $ created_at   : POSIXct, format: "2009-12-23 12:38:18" ...  
 $ retweet_count: int   28 33 13 5 7 4 2 4 1 22 ...  
 $ in_reply_to_user_id_str: chr  NA NA NA NA ...  
 $ favorite_count: int   12 6 11 3 6 5 2 10 4 30 ...  
 $ is_retweet   : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
```



scales

[Scale Functions for Visualization • scales \(r-lib.org\)](https://r-lib.org/packages/scales/)

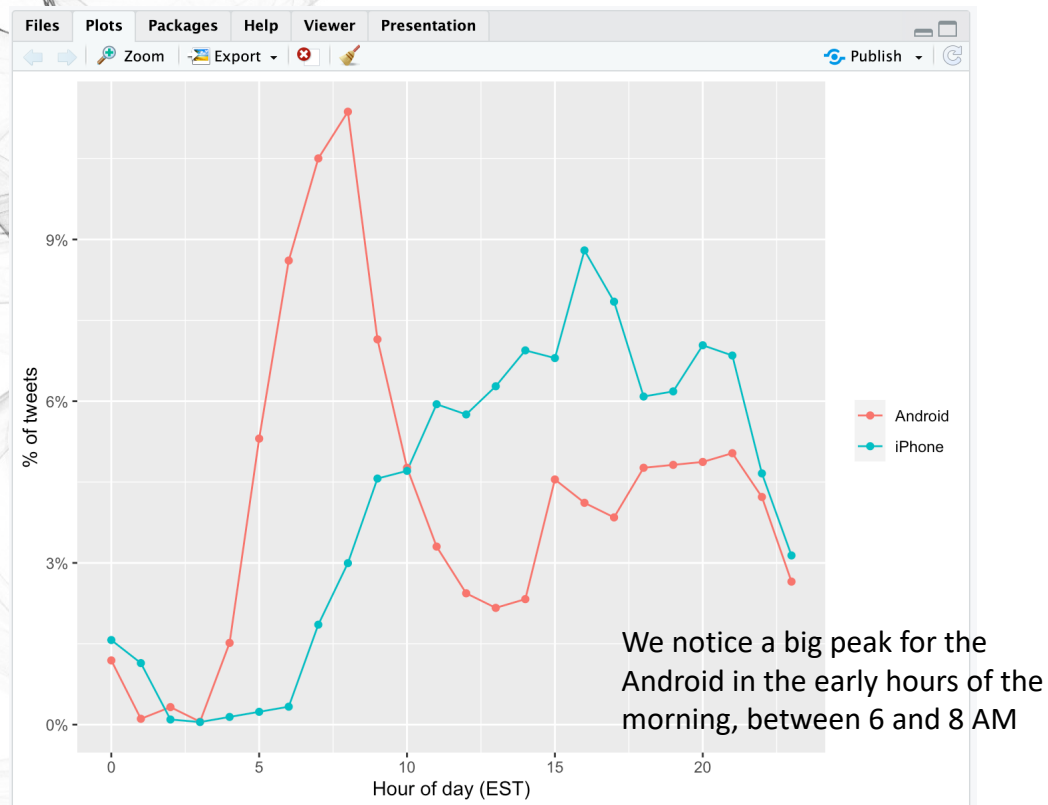
One of the most difficult parts of any graphics package is scaling, converting from data values to perceptual properties. The inverse of scaling, making guides (legends and axes) that can be used to read the graph, is often even harder! The scales packages provides the internal scaling infrastructure used by [ggplot2](https://ggplot2.tidyverse.org/), and gives you tools to override the default breaks, labels, transformations and palettes.

In general, we can extract data directly from Twitter using the rtweet package. However, in this case, a group has already compiled data for us and made it available at <https://github.com/bpb27/trump-tweet-archive>. We can get the data from their JSON API using a script like this

```
R 4.2.2 · ~/ ↵  
> trump_tweets |> count(source) |> arrange(desc(n)) |> head(10)  
      source      n  
1  Twitter Web Client 10718  
2  Twitter for Android  4652  
3  Twitter for iPhone  3962  
4      TweetDeck       468  
5  TwitLonger Beta     288  
6      Instagram      133  
7      Media Studio   114  
8      Facebook       104  
9      Twitter Ads     96  
10 Twitter for BlackBerry  78
```



```
R 4.2.2 · ~/
> campaign_tweets <- trump_tweets |>
+ extract(source, "source", "Twitter for (.*)") |>
+ filter(source %in% c("Android", "iPhone") &
+       created_at >= ymd("2015-06-17") &
+       created_at < ymd("2016-11-08")) |>
+ filter(!is_retweet) |>
+ arrange(created_at) |>
+ as_data_frame()
```



We are interested in what happened during the campaign, so for this analysis we will focus on what was tweeted between the day Trump announced his campaign and election day. We define the following table containing just the tweets from that time period. Note that we use `extract` to remove the Twitter for part of the source and filter out retweets

We can now use data visualization to explore the possibility that two different groups were tweeting from these devices. For each tweet, we will extract the hour, East Coast time (EST), it was tweeted and then compute the proportion of tweets tweeted at each hour for each device

```
R 4.2.2 · ~/
> campaign_tweets |>
+ mutate(hour = hour(with_tz(created_at, "EST"))) |>
+ count(source, hour) |>
+ group_by(source) |>
+ mutate(percent = n / sum(n)) |>
+ ungroup() |>
+ ggplot(aes(hour, percent, color = source)) +
+ geom_line() +
+ geom_point() +
+ scale_y_continuous(labels = percent_format()) +
+ labs(x = "Hour of day (EST)", y = "% of tweets", color = "")
```

Text as data

```
R 4.2.2 · ~/
> poem <- c("Roses are red,", "Violets are blue,",
+           "Sugar is sweet,", "And so are you.")
> example <- data_frame(line = c(1, 2, 3, 4),
+                        text = poem)
> example
# A tibble: 4 × 2
  line text
<dbl> <chr>
1     1 Roses are red,
2     2 Violets are blue,
3     3 Sugar is sweet,
4     4 And so are you.

> example |> unnest_tokens(word, text)
# A tibble: 13 × 2
  line word
<dbl> <chr>
1     1 roses
2     1 are
3     1 red
4     2 violets
5     2 are
6     2 blue
7     3 sugar
8     3 is
9     3 sweet
10    4 and
11    4 so
12    4 are
13    4 you
```

We will now study how the tweets differ when we compare Android to iPhone. To do this, we introduce the tidytext package

The main function needed to achieve this is `unnest_tokens`. **A token refers to a unit that we are considering to be a data point. The most common token will be words, but they can also be single characters, ngrams, sentences, lines, or a pattern defined by a regex.** The functions will take a vector of strings and extract the tokens so that each one gets a row in the new table

Text as data

```
R 4.2.2 · ~/
> i <- 3008
> campaign_tweets$text[i] |> cat()
Great to be back in Iowa! #TBT with @JerryJrFalwell joining me in Davenport-
this past winter. #MAGA https://t.co/A5IF0QHnic
> campaign_tweets[i,] |>
+ unnest_tokens(word, text) |>
+ pull(word)
[1] "great"      "to"         "be"         "back"
[5] "in"         "iowa"       "tbt"        "with"
[9] "jerryjrfalwell" "joining"    "me"         "in"
[13] "davenport"  "this"      "past"       "winter"
[17] "maga"       "https"     "t.co"       "a5if0qhnic"
```

Now let's look at an example from the tweets. We will look at tweet number 3008 because it will later permit us to illustrate a couple of points

```
R 4.2.2 · ~/
> links <- "https://t.co/[A-Za-z\\d]+"
> campaign_tweets[i,] |>
+ mutate(text = str_replace_all(text, links, "")) |>
+ unnest_tokens(word, text) |>
+ pull(word)
[1] "great"      "to"         "be"         "back"
[5] "in"         "iowa"       "tbt"        "with"
[9] "jerryjrfalwell" "joining"    "me"         "in"
[13] "davenport"  "this"      "past"       "winter"
[17] "maga"
```

A minor adjustment we want to make is to remove the links to pictures

Text as data

```
R 4.2.2 · ~/ ↵  
> tweet_words <- campaign_tweets |>  
+ mutate(text = str_replace_all(text, links, "")) |>  
+ unnest_tokens(word, text)
```

```
R 4.2.2 · ~/ ↵  
> tweet_words |>  
+ count(word) |>  
+ arrange(desc(n))  
# A tibble: 6,261 × 2  
  word      n  
  <chr> <int>  
1 the      2330  
2 to       1413  
3 and      1245  
4 in       1190
```

```
R 4.2.2 · ~/ ↵  
> tweet_words <- campaign_tweets |>  
+ mutate(text = str_replace_all(text, links, "")) |>  
+ unnest_tokens(word, text) |>  
+ filter(!word %in% stop_words$word )  
> tweet_words |> count(word) |> top_n(10, n) |> arrange(desc(n))  
# A tibble: 10 × 2  
  word      n  
  <chr> <int>  
1 trump2016 415  
2 hillary   407  
3 amp       325  
4 people    304  
5 makeamericagreatagain 298  
6 america   255  
7 clinton   240
```

We are now ready to extract the words for all our tweets

And we can now answer questions such as “what are the most commonly used words?” It is not surprising that these are the top words. The top words are not informative. The tidytext package has a database of these commonly used words, referred to as `stop_words`

If we filter out rows representing `stop_words`, we end up with a much more informative set of top 10 tweeted words

Text as data

Odds ratio

	Disease CASES	No disease CONTROLS
Exposed	a	b
unexposed	c	d

$$\text{Odds of exposure in CASES} = \frac{\# \text{ CASES with exposure}}{\# \text{ CASES without exposure}} = \frac{a}{c}$$

$$\text{Odds of exposure in CONTROLS} = \frac{\# \text{ CONTROLS with exposure}}{\# \text{ CONTROLS without exposure}} = \frac{b}{d}$$

$$\frac{\text{Odds of exposure in CASES}}{\text{Odds of exposure in CONTROLS}} = \frac{a/c}{b/d} = \frac{ad}{bc} = OR$$

Now that we have all our words in a table, along with information about what device was used to compose the tweet they came from, we want to know if a word is more likely to come from an Android tweet or an iPhone tweet. **Odds ratio (OR)** is a summary statistic useful for quantifying the strength of the association between two events, A and B

R 4.2.2 · ~/

```
> c <- 0.499
> android_iphone_or <- tweet_words |>
+ count(word, source) |>
+ pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
+ mutate(or = (Android + c) / (sum(Android) - Android + c) /
+           ( (iPhone + c) / (sum(iPhone) - iPhone + c)))
```

For each word and a device, we compute an OR. To avoid divided by zero, we use the 0.499 trick

Text as data

Already getting insights!

Here are the highest odds ratios for Android

```
R 4.2.2 · ~/ 
> android_iphone_or |> arrange(desc(or)) |> head(20)
# A tibble: 20 × 4
```

	word <chr>	Android <int>	iPhone <int>	or <dbl>
1	a.m	65	0	114.
2	p.m	39	0	68.7
3	00	94	1	54.9
4	mails	22	0	39.1
5	poor	13	0	23.5
6	t.v	13	0	23.5
7	v.p	13	0	23.5
8	poorly	12	0	21.7
9	bosses	11	0	20.0
10	cbsnews	11	0	20.0
11	turnberry	11	0	20.0
12	angry	10	0	18.2
13	write	10	0	18.2
14	brexit	9	0	16.5
15	defend	9	0	16.5
16	funny	9	0	16.5
17	playing	9	0	16.5
18	pols	9	0	16.5
19	sources	9	0	16.5
20	stupid	9	0	16.5

Here are the highest odds ratios for iPhone

```
R 4.2.2 · ~/ 
> android_iphone_or |> arrange(or) |> head(20)
# A tibble: 20 × 4
```

	word <chr>	Android <int>	iPhone <int>	or <dbl>
1	makeamericagreatagain	0	298	0.00142
2	americafirst	0	71	0.00602
3	draintheswamp	0	63	0.00678
4	trump2016	3	412	0.00714
5	votetrump	0	56	0.00762
6	join	1	157	0.00816
7	imwithyou	0	51	0.00837
8	crookedhillary	0	30	0.0141
9	fitn	0	30	0.0141
10	gopdebate	0	30	0.0141
11	trumppence16	0	28	0.0151
12	7pm	0	25	0.0169
13	bigleaguetruth	0	22	0.0192
14	iacaucus	0	21	0.0201
15	icymi	0	21	0.0201
16	officers	0	21	0.0201
17	30pm	0	20	0.0211
18	3pm	0	19	0.0221
19	trumptrain	0	18	0.0233
20	supertuesday	0	17	0.0247

Text as data

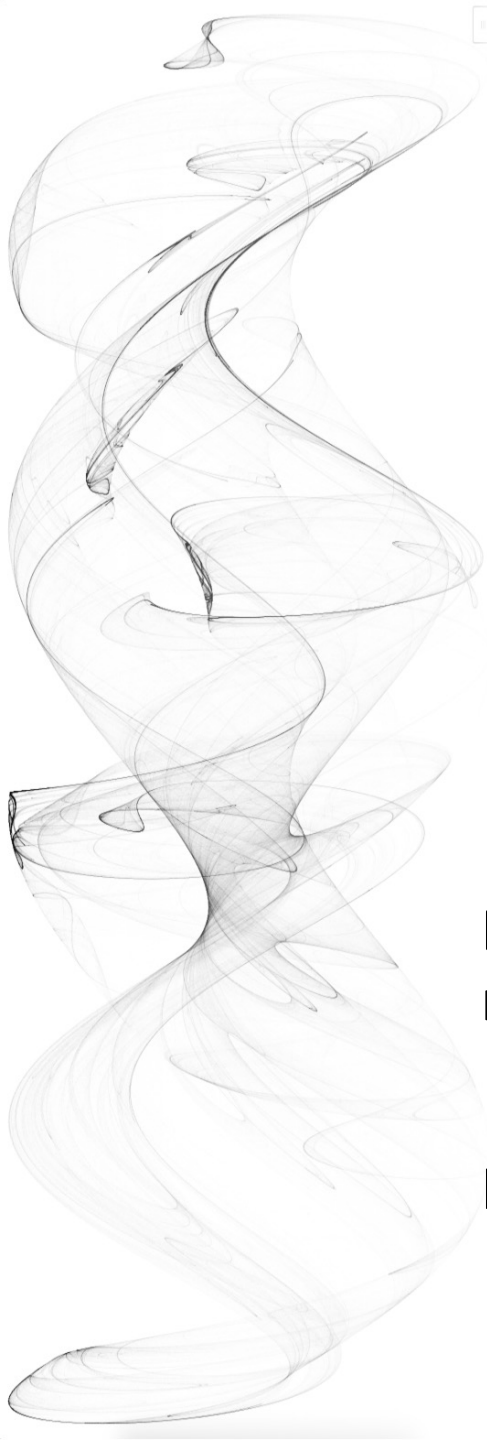
More frequently used words should be more convincing, we can impose a filter based on the total frequency like this

Android

```
R 4.2.2 · ~/ ↵
> android_iphone_or |>
+ filter(Android+iPhone > 100) |>
+ arrange(desc(or)) |> head(20)
# A tibble: 20 × 4
  word      Android iPhone   or
  <chr>      <int>  <int> <dbl>
1 bad          104     26  3.43
2 crooked       156     49  2.76
3 cnn           116     37  2.70
4 ted           86     28  2.64
5 interviewed   76     25  2.61
6 media         78     26  2.57
7 hillary       290    117  2.16
8 cruz          115     46  2.16
9 win           74     30  2.12
10 president     84     35  2.07
11 foxnews        77     39  1.70
12 debate         80     42  1.64
13 people        195    109  1.55
14 country        67     38  1.52
15 night          70     40  1.51
16 time           90     55  1.42
17 amazing        66     42  1.36
18 crowd          60     43  1.21
19 enjoy          69     52  1.15
20 clinton       136    104  1.13
```

iPhone

```
R 4.2.2 · ~/ ↵
+ filter(Android+iPhone > 100) |>
+ arrange(or) |> head(20)
# A tibble: 20 × 4
  word      Android iPhone   or
  <chr>      <int>  <int> <dbl>
1 makeamericagreatagain     0    298 0.00142
2 trump2016                   3    412 0.00714
3 join                        1    157 0.00816
4 tomorrow                   24    101 0.208
5 amp                         64    261 0.211
6 vote                        46     67 0.596
7 america                    114    141 0.699
8 tonight                     70     84 0.722
9 iowa                        62     65 0.826
10 poll                       117    103 0.984
11 trump                      112     92 1.05
12 clinton                    136    104 1.13
13 enjoy                       69     52 1.15
14 crowd                       60     43 1.21
15 amazing                     66     42 1.36
16 time                        90     55 1.42
17 night                       70     40 1.51
18 country                     67     38 1.52
19 people                      195    109 1.55
20 debate                      80     42 1.64
```


A decorative graphic on the left side of the slide consisting of multiple overlapping, wavy, grey lines that create a sense of movement and depth.

Insights

Since the frequently used words do not overlap well, it's likely that the iPhone tweets and the Android tweets are from different tweeters



Todd Vaziri 
@tvaziri

Every non-hyperbolic tweet is from iPhone (his staff).

Every hyperbolic tweet is from Android (from him).

But Todd's conjecture was non-hyperbolic against hyperbolic tweets, we need a bit of sentiment analysis

In sentiment analysis, we assign a word to one or more “sentiments”

Sentiment analysis

[get_sentiments function - RDocumentation](#)

The Bing lexicon divides words into positive and negative sentiments

```
R 4.2.2 · ~/ ↗  
> get_sentiments("bing")  
# A tibble: 6,786 × 2  
  word      sentiment  
  <chr>    <chr>  
1 2-faces    negative  
2 abnormal  negative  
3 abolish   negative  
4 abominable negative  
5 abominably negative  
6 abominate  negative  
7 abomination negative  
8 abort      negative  
9 aborted    negative  
10 abortions negative  
# ... with 6,776 more rows  
# i Use `print(n = ...)` to see more rows
```

```
R 4.2.2 · ~/ ↗  
> library(tidytext)  
> library(textdata)
```

The AFINN lexicon assigns a score between -5 and 5, with -5 the most negative and 5 the most positive

```
R 4.2.2 · ~/ ↗  
> get_sentiments("afinn")  
# A tibble: 2,477 × 2  
  word      value  
  <chr>    <dbl>  
1 abandon    -2  
2 abandoned  -2  
3 abandons   -2  
4 abducted   -2  
5 abduction  -2  
6 abductions -2  
7 abhor      -3  
8 abhorred   -3  
9 abhorrent  -3  
10 abhors    -3  
# ... with 2,467 more rows  
# i Use `print(n = ...)` to see more rows
```

The first step in sentiment analysis is to assign a sentiment to each word. We will use the tidytext package and the textdata package

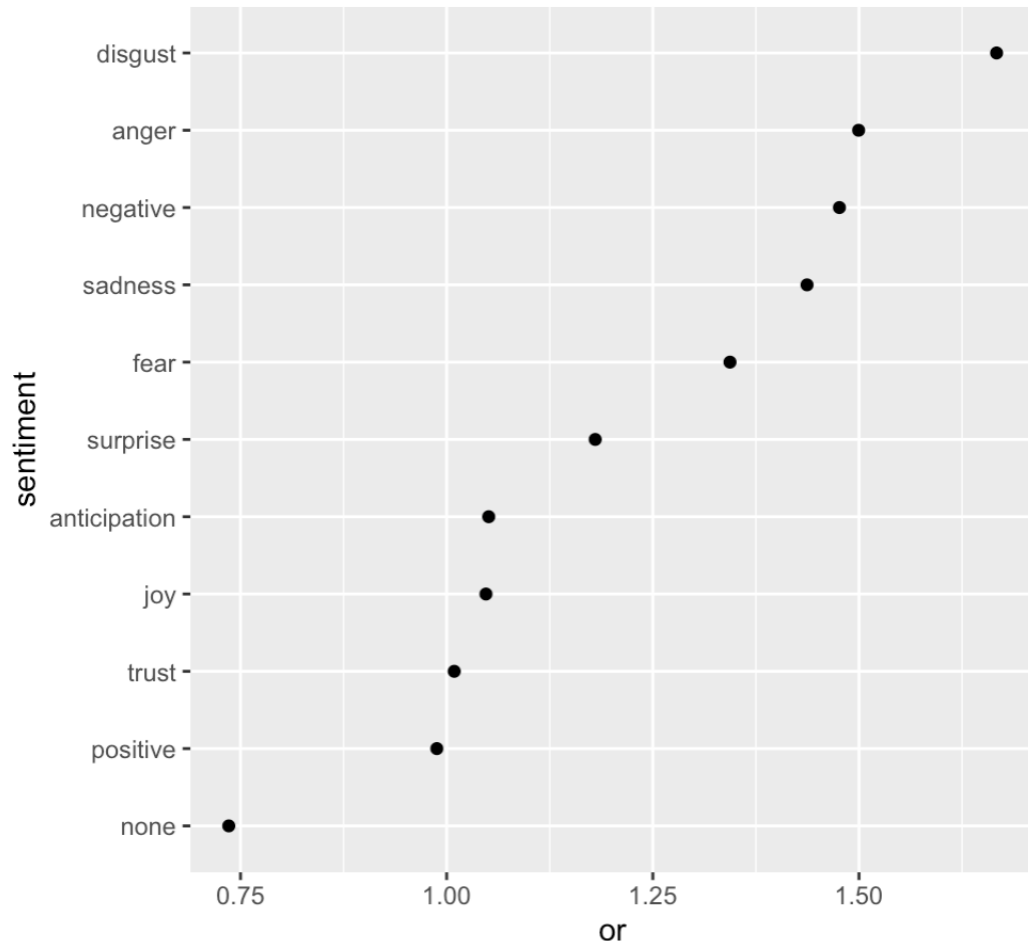
The nrc lexicon is suitable for social media posts

```
R 4.2.2 · ~/ ↗  
> nrc |> count(sentiment)  
# A tibble: 10 × 2  
  sentiment      n  
  <chr>    <int>  
1 anger    1245  
2 anticipation 837  
3 disgust    1056  
4 fear      1474  
5 joy        687  
6 negative   3316  
7 positive   2308  
8 sadness    1187  
9 surprise    532  
10 trust     1230
```

Sentiment analysis

R 4.2.2 · ~/

```
> nrc <- get_sentiments("nrc")
```



For our analysis, we will use the nrc lexicon

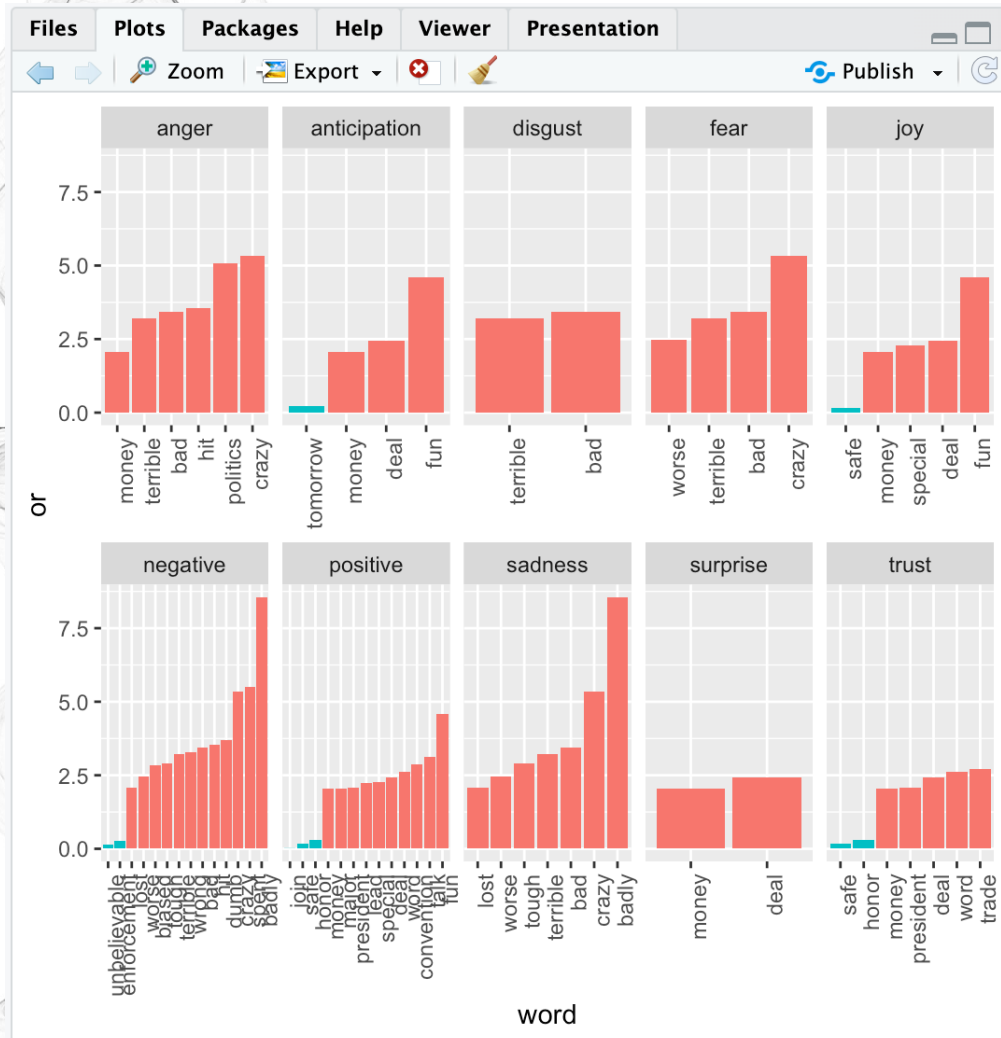
R 4.2.2 · ~/

```
> tweet_words |>  
+ left_join(nrc, by = "word") |>  
+ select(source, word, sentiment) |>  
+ count(source, sentiment) |>  
+ pivot_wider(names_from = "source", values_from = "n") |>  
+ mutate(sentiment = replace_na(sentiment, replace = "none")) |>  
+ mutate(Android = Android / (sum(Android) - Android),  
+         iPhone = iPhone / (sum(iPhone) - iPhone),  
+         or = Android/iPhone) |>  
+ mutate(sentiment = reorder(sentiment, or)) |>  
+ ggplot(aes(y = sentiment, x = or)) +  
+ geom_point()
```

We see that the disgust, anger, negative, sadness, and fear sentiments are associated with the Android in a way that is hard to explain by chance alone. Words not associated to a sentiment were strongly associated with the iPhone source, which is in agreement with the original claim about hyperbolic tweets

Sentiment analysis

If we are interested in exploring which specific words are driving these differences



```
R 4.2.2 · ~/
```

```
> android_iphone_or |> inner_join(nrc, by = "word") |>  
+ mutate(sentiment = factor(sentiment, levels = count(nrc,sentiment)$sentiment)) |>  
+ filter(Android + iPhone > 20, or > 2 | or < 0.5) |>  
+ mutate(word = reorder(word, or)) |>  
+ ggplot(aes(word, or, fill = or < 1)) +  
+ facet_wrap(~sentiment, scales = "free_x", nrow = 2) +  
+ geom_bar(stat="identity", show.legend = FALSE) +  
+ theme(axis.text.x = element_text(angle = 90, hjust = 1))
```

This is just a simple example of the many analyses one can perform with tidytext. To learn more, we again recommend the Tidy Text Mining book at <https://www.tidytextmining.com/>



Summary

- We examined a large collection of tweets through:
 - Looking at relevant metadata:
 - When were they posted?
 - Looking at the tweet contents:
 - Tokenize all tweets and remove irrelevant words
 - Calculate the occurrences and odds ratio of words in different devices
 - Sentiment analysis of the contents using word-to-sentiment mapping
- To learn more:
 - `tidytext` tutorial:
 - <https://www.tidytextmining.com/tidytext>
 - Some more twitter datasets to play around with:
 - <https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment>
 - <https://www.kaggle.com/datasets/kazanova/sentiment140>
 - <https://www.kaggle.com/datasets/gpreda/covid19-tweets>